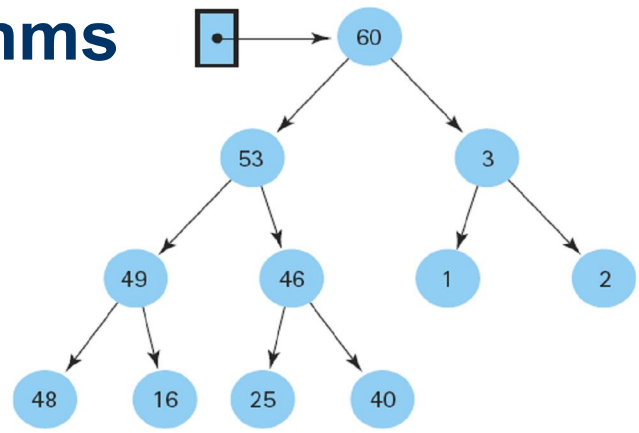
Data Structures & Algorithms



تعريف المخططات What is a Graph ?

- **Definition:** A graph is a collection (nonempty set) of vertices and edges

A graph G = (V,E) is composed of:

V: set of vertices

E: set of edges connecting the vertices in V

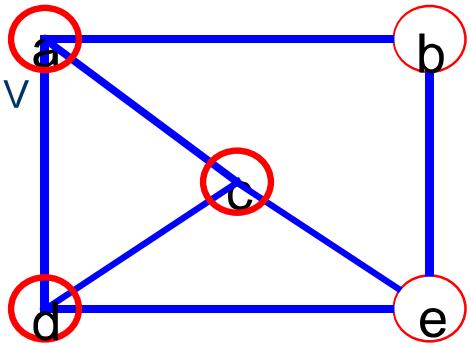
Vertices: can have names and properties

Edges: connect two vertices,

can be labeled,

can be directed

- An edge e = (u,v) is a pair of vertices

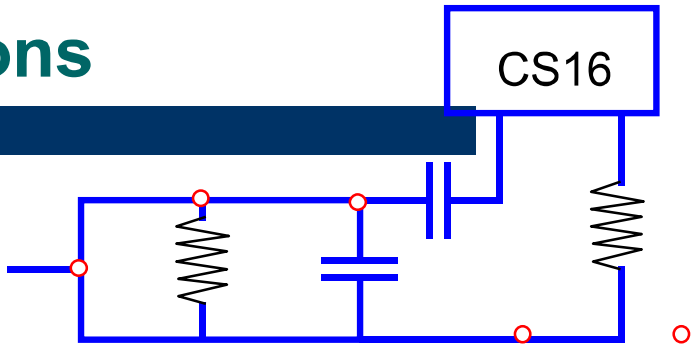


V= {a,b,c,d,e}
E= {(a,b),(a,c),(a,d),(b,e),(c,d),(c,e),(d,e)}

التطبيقات Applications

CS16

- electronic circuits



- networks (roads, flights, communications)

المخططات الموجه والغير موجه Directed vs. Undirected Graph

- An undirected graph is one in which the pair of vertices in a edge is unordered, $(v_0, v_1) = (v_1, v_0)$
- A directed graph is one in which each edge is a directed pair of vertices, $<v_0, v_1> != <v_1, v_0>$

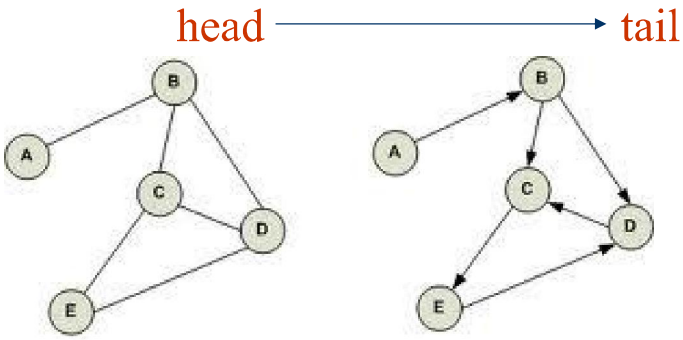


Fig 1. Undirected Graph Fig 2. Directed Graph

Terminology مصطلحات : *Degree of a Vertex* درجة القمة

- The degree of a vertex is the number of edges incident to that vertex
- For directed graph,
  - The out-degree of a vertex *v* is the number of edges that have *v* as the head
  - the in-degree of a vertex *v* is the number of edges that have *v* as the tail

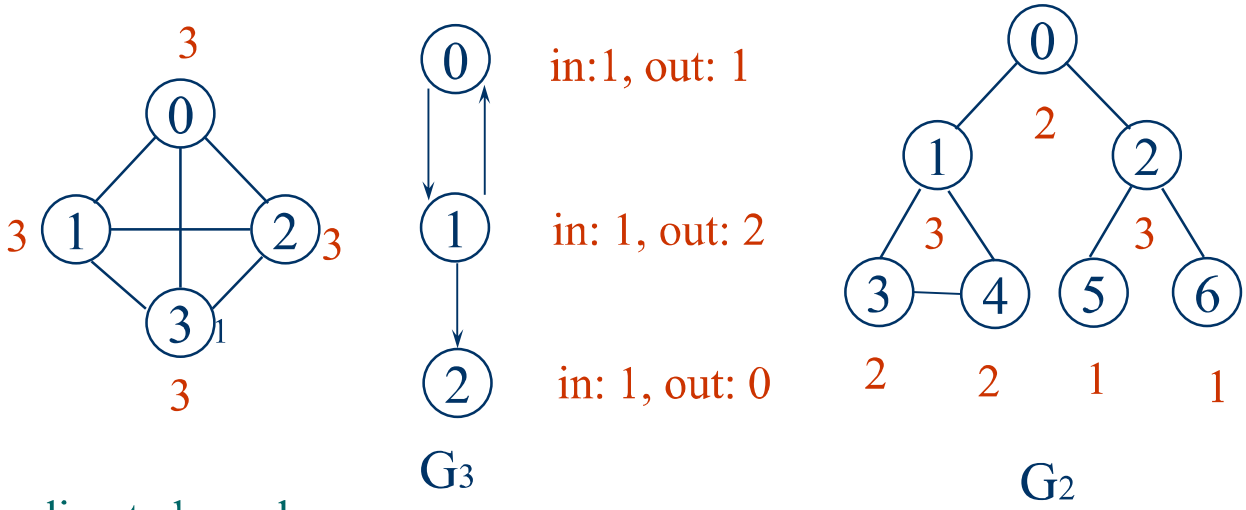
    A node with indegree 0 is a ***root***.

if d_i is the degree of a vertex i in a graph G with n vertices and e edges, the number of edges is

$$e = \left(\sum_{0}^{n-1} d_i\right) / 2$$

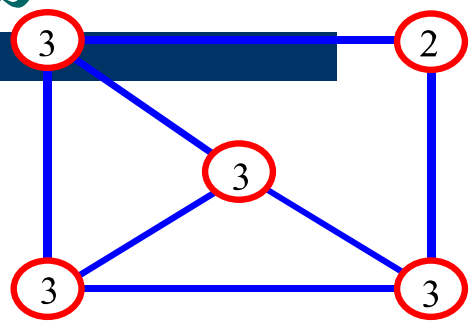
Why? Since adjacent vertices each count the adjoining edge, it will be counted twice

Examples



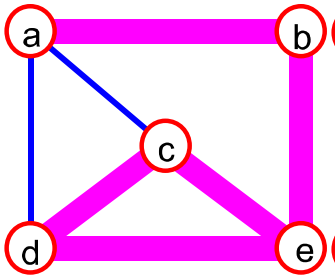
directed graph
in-degree
out-degree

Terminology: Path الطريق

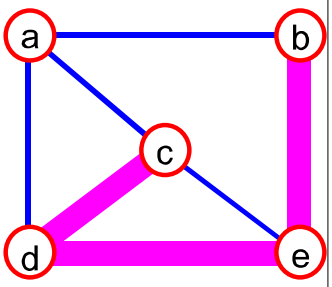


- path: sequence of vertices $v_1, v_2, \ldots v_k$ such that consecutive vertices v_i and v_{i+1} are adjacent.

- The *length* of the path is N-1.



a b e d c

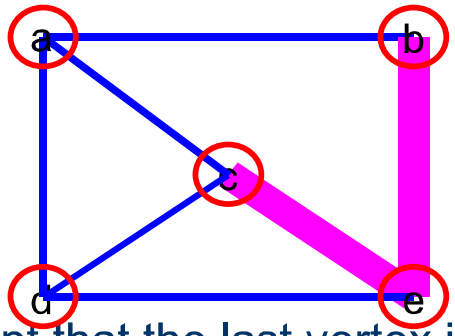


b e d c

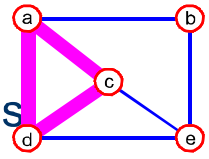
More Terminology

- simple path الطريق البسيط: no repeated vertices

b e c



- Cycle الحلقة: simple path, except that the last vertex is the same as the first vertex
- A *cycle* is a path such that $v_1 = v_N$
  - An *acyclic* لا حلقي graph has no cycles



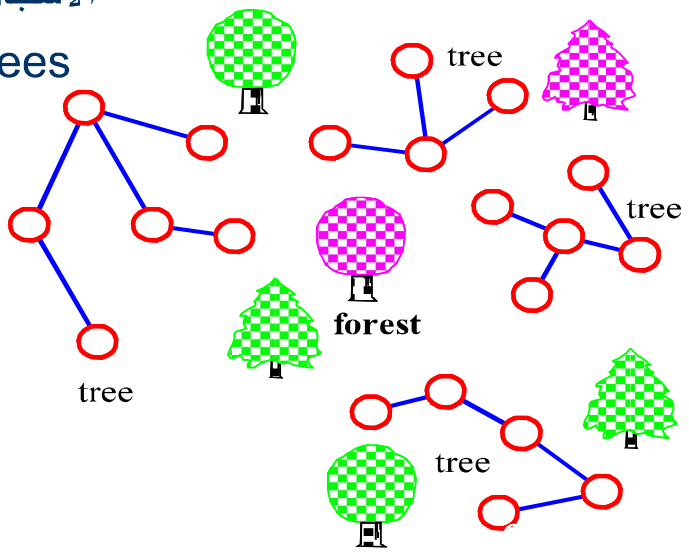
a c d a

More…

- tree - connected graph without cycles

الاشجار مجموعة جزئية من المخططات

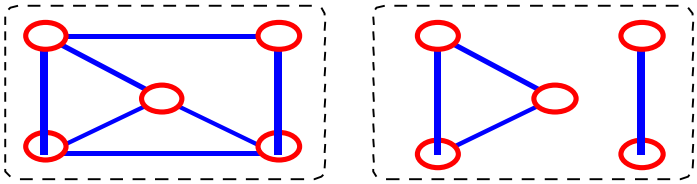
- forest - collection of trees



المخططات المتصلة والغير متصلة Connected graph and Disconnected graph

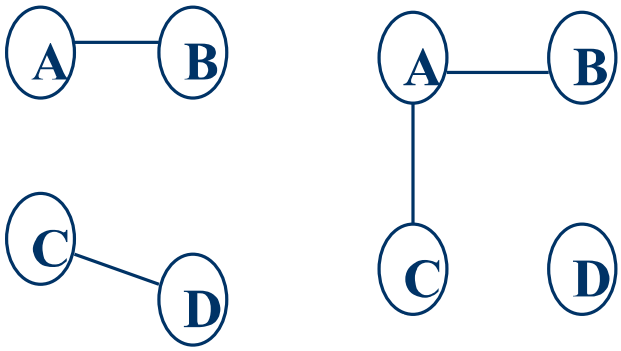
connected graph: There is a path between each two vertices

نستطيع ان نصل الى أي راس بواسطة أي راس اخر سواء بطرق مباشرة او غير مباشرة



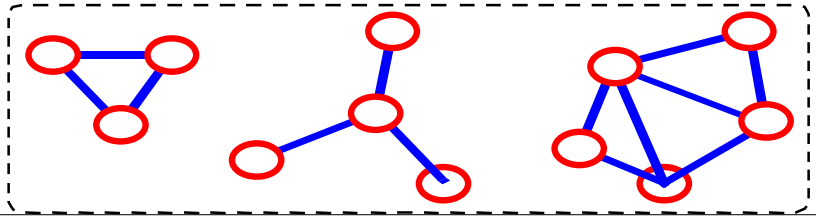
Disconnected graph : There are at least two vertices not connected by a path.

Examples of disconnected graphs:

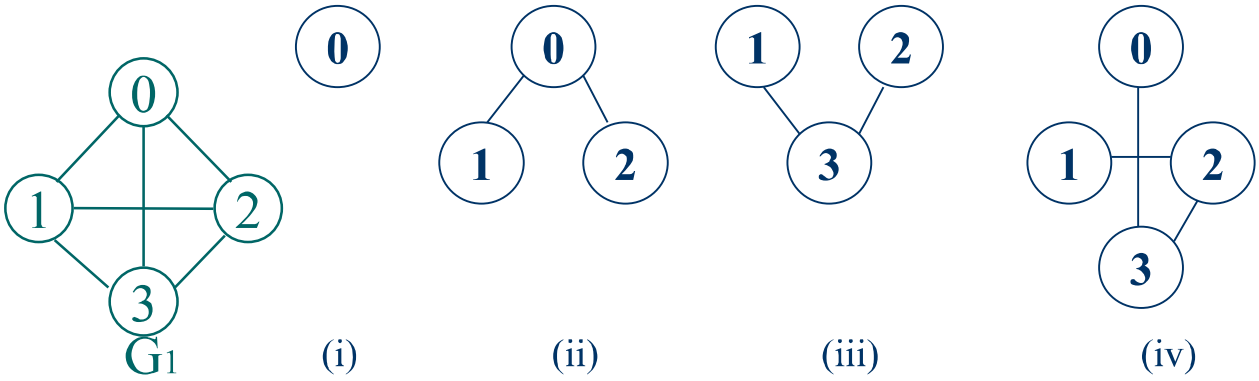


Even More Terminology

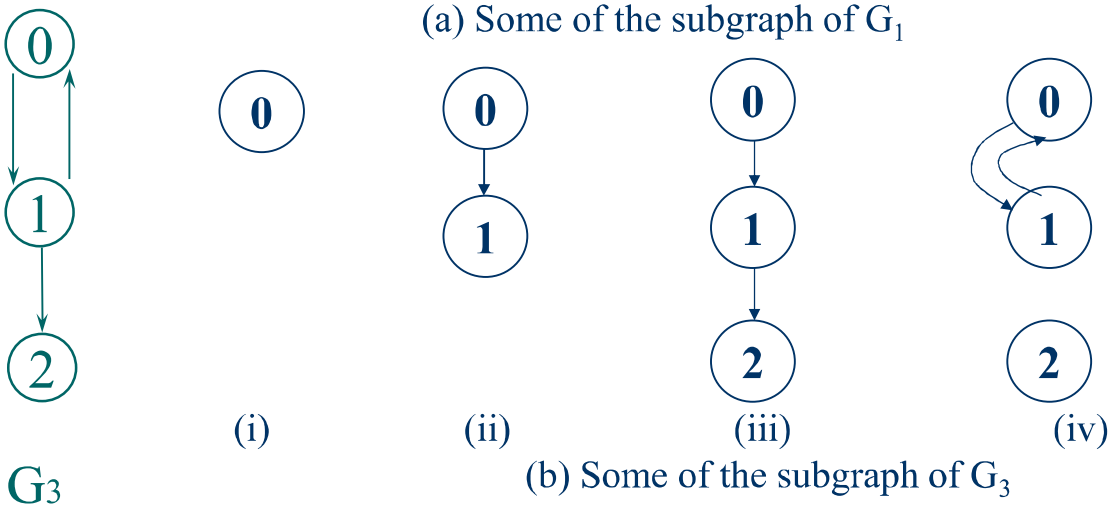
- subgraph: subset of vertices and edges forming a graph
- connected component: maximal connected subgraph. E.g., the graph below has 3 connected components.



Subgraphs Examples



(a) Some of the subgraph of G_1



(b) Some of the subgraph of G_3

Connectivity

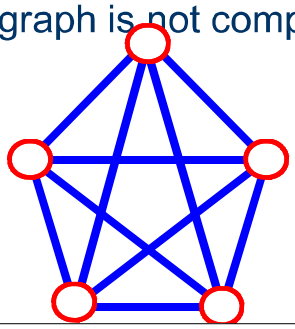
- **A complete graph المخطط المكتمل**: one in which all pairs of vertices are adjacent كل راس مرتبط بجميع الرؤوس في المخطط بطريقة مباشرة
- *How many total edges in a complete graph?*

كم عدد الحواف بالمخطط المكتمل؟ باستخدام عدد الرؤوس

Let **n** = #vertices, and **m** = #edges

- Each of the n vertices is incident to **n-1** edges, however, we would have counted each edge twice! Therefore, intuitively, m = **n**(**n** -1)/2.

- Therefore, if a graph is not complete, m < **n**(**n** -1)/2



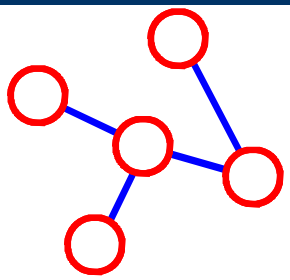
$\mathbf{n} = 5$
$\mathbf{m} = (5 * 4)/2 = 10$

More Connectivity

n = #vertices

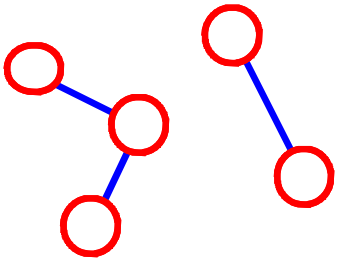
m = #edges

- For a tree **m** = **n** - 1



$\mathbf{n} = 5$
$\mathbf{m} = 4$

If **m** < **n** - 1, G is not connected



$\mathbf{n} = 5$
$\mathbf{m} = 3$

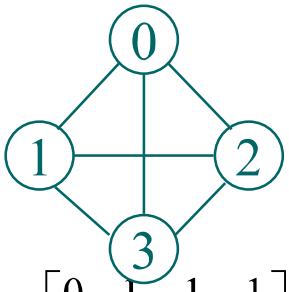
Graph Representations تمثيل المخططات

- Adjacency Matrix المصفوفة
- Adjacency Lists القوائم

Adjacency Matrix

- Let G=(V,E) be a graph with n vertices.
- The adjacency matrix of G is a two-dimensional n by n array, say adj_mat
- If the edge (v_i, v_j) is in E(G), adj_mat[i][j]=1
- If there is no such edge in E(G), adj_mat[i][j]=0
- The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric

Examples for Adjacency Matrix

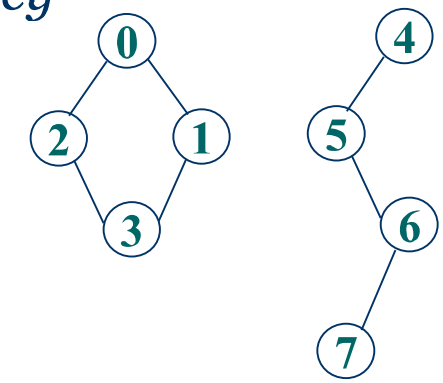


$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

G_1

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

G_2



$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

G_4

متماثلة حول القطر symmetric

undirected: $n^2/2$
directed: n^2

Merits of Adjacency Matrix مزايا

- From the adjacency matrix, to determine the connection of vertices is easy

تحديد الوصلات للقمم اسهل

- The degree of a vertex is $\sum_{j=0}^{n-1} adj_mat[i][j]$

درجة الراس Vi هو مجموع عناصر الصف i

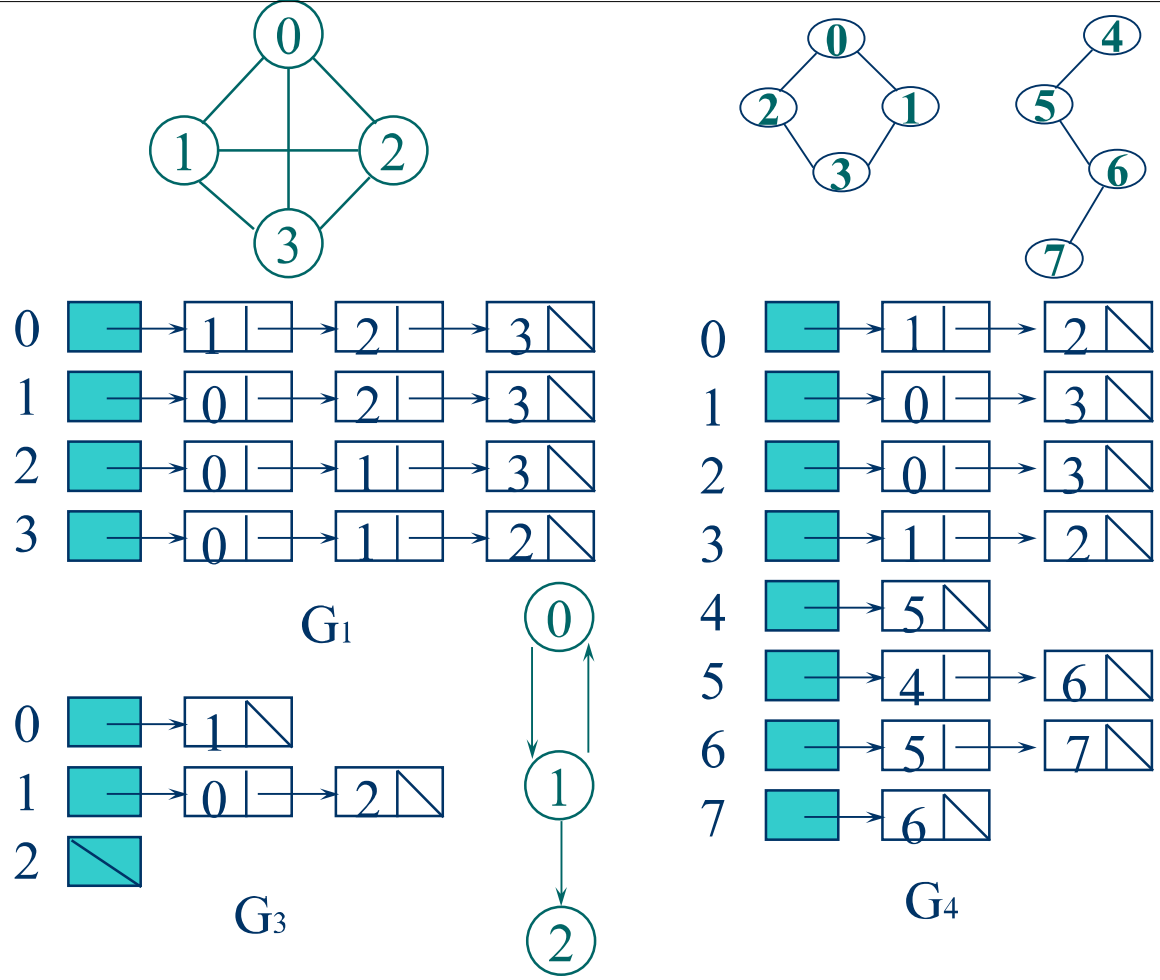
- For a digraph (= directed graph), the row sum is the out_degree, while the column sum is the in_degree

للمخططات الموجه out-degree= مجموع الصفوف

In-degree = مجموع الاعمدة

$$outd(vi) = \sum_{j=0}^{n-1} A[i,j]$$

$$ind(vi) = \sum_{j=0}^{n-1} A[j,i]$$



An undirected graph with n vertices and e edges ==> n head nodes and 2e list nodes

Some Operations

- degree of a vertex in an undirected graph
  - # of nodes in adjacency list
- # of edges in a graph
  - determined in O(n+e)
- out-degree of a vertex in a directed graph
  - # of nodes in its adjacency list
- in-degree of a vertex in a directed graph
  - traverse the whole data structure

Adjacency Lists vs. Matrix

- **Adjacency Lists**
  - More compact احكاما than adjacency matrices if graph has few edges
  - Requires more time to find if an edge exists

- اكثر احكاما واقل استهلاك للمساحة خصوصا اذا كان عدد الحواف قليل
- تستهلك زمن اكثر لايجاد حافة ما

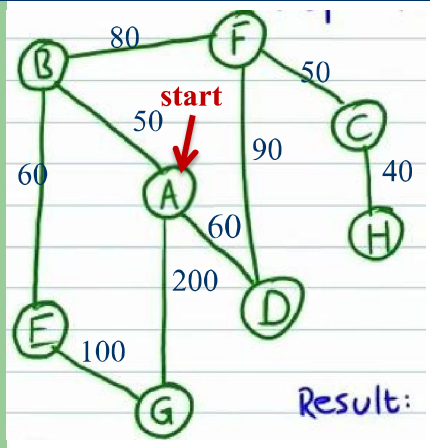
- **Adjacency Matrix**
  - Always require n^2 space
    - This can waste a lot of space if the number of edges are sparse
  - Can quickly find if an edge exists

- دائما تحتاج الى مساحة n^2 بالتالي تستهلك مساحة اكبر يمكن ان تضيع بلا جدوى خصوصا اذا كانت الحواف متناثرة لذلك ا اما اذا كان المخطط مكتمل او شبه مكتمل يكون التمثل بالمصفوفة المترابطة افضل
- تستهلك زمن اقل لايجاد حافة ما

استعراض المخططاتGraph Traversal

- <u>Problem:</u> Search for a certain node or traverse all nodes in the graph يستخدم للبحث عن نود معينة او استعراض جميع النود بالمخطط
- Depth First Search البحث بالعمق
  - Once a possible path is found, continue the search until the end of the path
- Breadth First Search البحث بالعرض
  - Start several paths at a time, and advance in each one step at a time

Depth-First Search

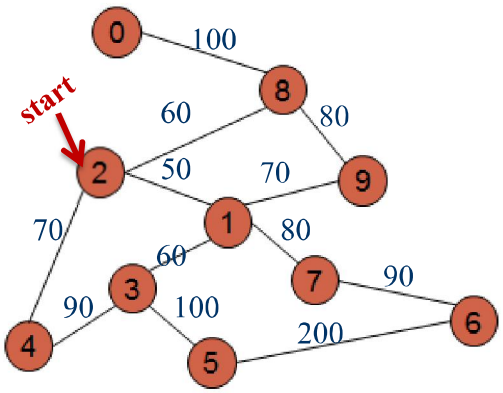


Result: A B E G F C H D

vertex	Initial	Visited?	Who?
A	F	T	-
B	F	T	A
C	F	T	F
D	F	T	F
E	F	T	B
F	F	T	B
G	F	T	E
H	F	T	C

Path (H) = ?
Path (E) = ?

Print out all the keys in DFS traversal :



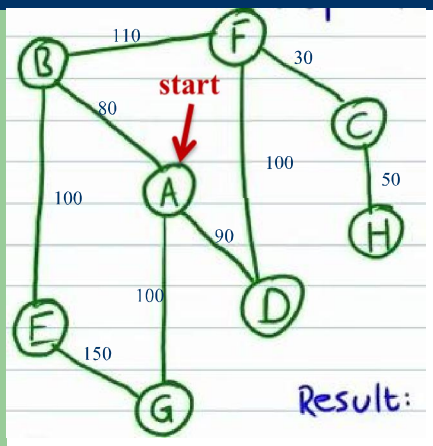
Result :
2 1 3 4 5 6 7 9 8 0

Path (6) = ?
Path (3) = ?

DFS Applications :

Solving puzzles with only one solution, such as mazes. (DFS can be adapted to find all solutions to a maze by only including nodes on the current path in the visited set.)

Breadth first search :

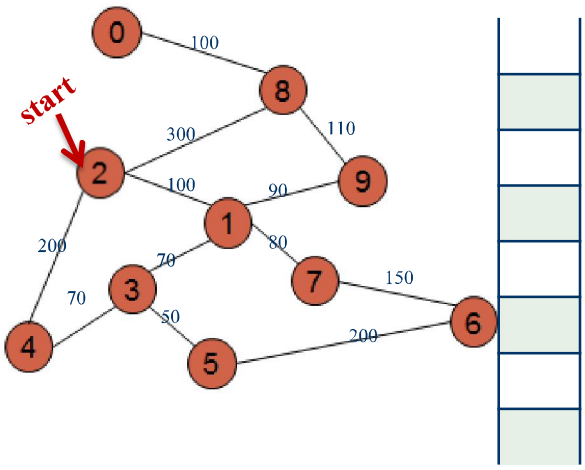


vertex	Initial	Visited?	Who?
A	F	T	-
B	F	T	A
C	F	T	F
D	F	T	A
E	F	T	B
F	F	T	B
G	F	T	A
H	F	T	C

Result: A B D G E F C H

Path (H) = ?
Path (E) = ?

print out all key in BFS traversal :



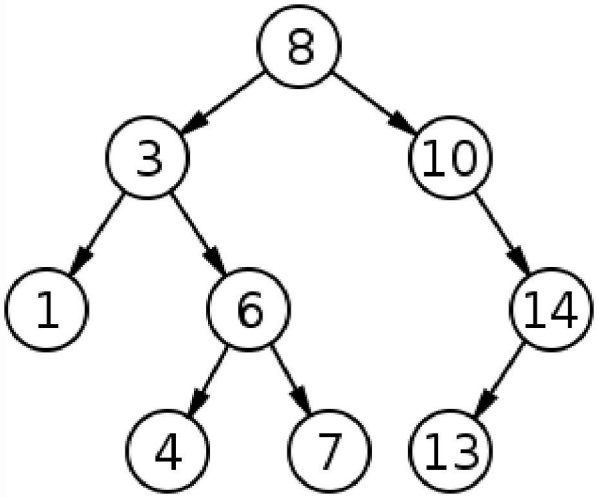
Result :
2 1 4 8 3 7 9 0 5 6

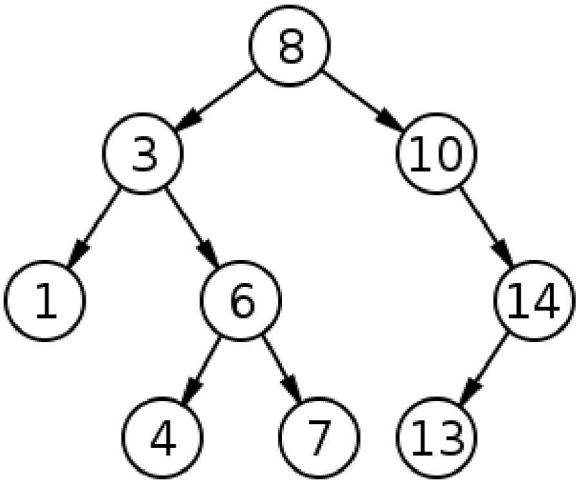
Path (6) = ?
Path (3) = ?

BFS applications

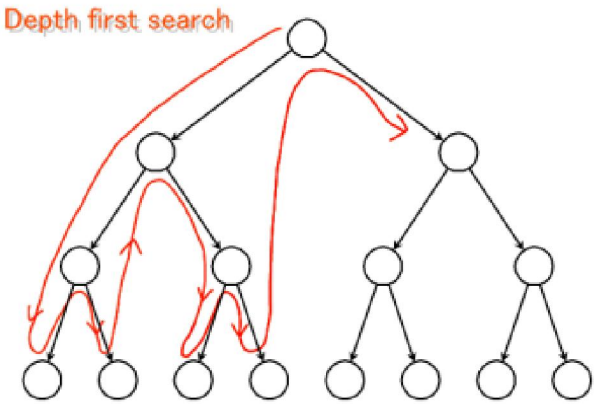
- Finding the <u>shortest path</u> between two nodes *u* and *v*

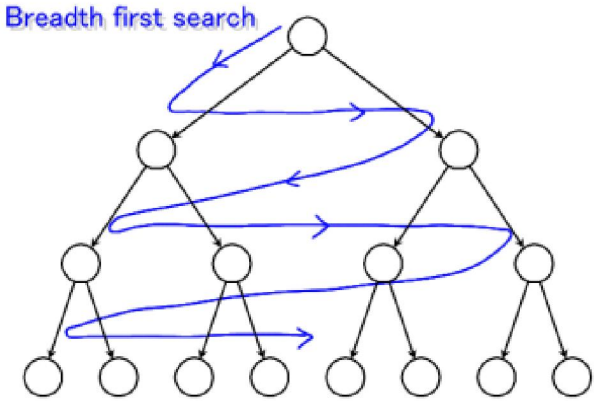
BFS & DFS for tree :





BFS & DFS for tree :





- Both take time: O(V+E)

End